Project Progress Monitoring System – Full Technical Specification (React + ASP.NET Core)

1. Overview

The Project Progress Monitoring System is a web and mobile application that enables organizations to plan projects, assign work, and continuously monitor progress against time, cost, and scope baselines. It centralizes project data (projects, tasks, milestones, time logs, and status updates) and provides dashboards and reports for managers and stakeholders.[1][2][3][4]

The solution consists of a React web client, a React Native mobile client, and an ASP.NET Core Web API backend implemented using a Clean Architecture style with Domain, Application, Infrastructure, and Presentation layers, all connected to a relational database such as SQL Server or PostgreSQL. The backend exposes secure RESTful APIs for all entities and business operations, and both clients consume these APIs for project setup, execution tracking, collaboration, and reporting.[5][6][7][8][^1]

2. System goals and scope

The primary goal is to provide real-time visibility into project health (schedule, effort, and status) so that managers can identify delays, risks, and bottlenecks early and take corrective actions. The system targets IT, construction, and government projects where distributed teams and frequent status updates make manual tracking in spreadsheets inefficient.[2][4][^1]

In scope:

- Multi-project management for one or more companies/clients.
- Web portal for managers and coordinators.
- Mobile app for on-field staff and team members to update tasks and log time.
- Core tracking of projects, milestones, tasks, time logs, progress updates, and attachments.
- Dashboards and standard reports for monitoring and control.

3. Actors and user roles

Common roles in project tracking tools include administrators, project managers, team members, and clients/stakeholders.[3][2]

- **System Administrator** – Manages global configuration, user accounts, roles, and reference data.
- **Project Manager** – Creates and manages projects, milestones, tasks, assignments, and approvals.

- **Team Member** – Views assigned tasks, submits progress updates and time logs, and uploads attachments.
- **Client / Stakeholder (optional)** – Read-only access to selected projects and reports.

4. Detailed feature list

4.1 Authentication and user management

- User registration (admin-provisioned or self-registration controlled by policy).
- Secure login with email/username and password.
- Password reset (email link / OTP-based, depending on deployment).
- Role-based access control (Admin, ProjectManager, Member, Client).
- User profile management (name, contact, designation, avatar).
- Account activation/deactivation by admin.

These capabilities are typical in enterprise project and monitoring systems for controlled access to project data.[^1]

4.2 Company and client management

- Create and manage companies/clients (name, industry, address, contact person, contact details).
- Associate projects with one company.
- View list of projects under a company with high-level metrics (count of active projects, completion percentage, total budget vs. actual spend – if cost tracking is implemented).

Project monitoring products usually support multi-client or multi-department configuration to cover enterprise scenarios.[^1]

4.3 Project management features

- Create projects with attributes:
  - Name, description, company, start and end dates.
  - Budget and currency (optional).
  - Status (Planned, Active, On Hold, Completed, Cancelled).
  - Priority (High, Medium, Low).
  - Project manager and default team.
- Edit/close/cancel projects with audit trail.
- Assign multiple members to a project with roles (developer, site engineer, QA, etc.) and allocation percentage.
- Maintain project-level documents (contracts, SOW, design docs) via attachments.
- Project overview page showing:

    - % completion (derived from tasks/milestones).
    - Number of open/closed tasks.
    - Upcoming milestones and overdue items.

These features align with standard project management software capabilities described in monitoring and control guidance.[4][2]

4.4 Milestone management

- Define milestones for each project with title, description, due date, and status.
- Attach tasks to milestones to group related work.
- Track milestone completion percentage based on underlying tasks.
- Highlight overdue / at-risk milestones on the dashboard.

Milestone tracking is a common technique for monitoring and controlling projects, especially for long-running or construction projects.[^4]

4.5 Task and assignment management

- Create tasks under a project (and optionally under a milestone) with:
    - Title, detailed description.
    - Start and end dates / planned effort.
    - Status (To Do, In Progress, Blocked, Done).
    - Priority (Critical, High, Medium, Low).
    - Estimated hours.
- Assign one or more team members to each task.
- Support task dependencies (predecessor tasks) for schedule insight.
- Kanban board view (columns for To Do, In Progress, Blocked, Done).
- List view with filtering and sorting (by assignee, status, priority, dates).

Task lists and Kanban boards are standard tools in project tracking software for visualizing and controlling work.[^2]

4.6 Progress updates

- Allow users to submit progress updates at task or project level.
- For each update capture:
    - Date/time.
    - Percent complete (0–100).
    - Status summary / comment.
    - Issues or risks.
- Calculate derived values:

    - Overall project % complete (aggregation of tasks/milestones).
    - Trend of progress over time (for charts).
- Show history timeline of updates per task and per project.

Centralized progress updates and history views are core to monitoring and control activities in project management.[3][4]

4.7 Time tracking and worklogs

- Team members log time spent on tasks:
    - Task, user, date, hours, and description.
- Daily/weekly timesheet view for users.
- Manager view of total hours per project, per user, and per period.
- Optional rule checks (e.g., max hours per day) and approvals.

Time logging is a common requirement in project tracking to relate effort to schedule and cost performance.[^3]

4.8 Attachments and document management

- Upload files at project and task level (design documents, site photos, approvals, etc.).
- Store metadata: uploader, upload date, file name, and size.
- Preview or download from web and mobile apps.

Project monitoring solutions, particularly in government and construction, often integrate photo and document uploads as part of progress evidence.[^1]

4.9 Dashboards and reports

- Project dashboard for a manager:
    - Card for each project with health indicators.
    - Charts: tasks by status, milestones on track vs. delayed, completion over time.
- Organization / company dashboard:
    - Active vs. completed projects.
    - Projects by status and priority.
- Standard reports:
    - Project status report.
    - Task summary report by user, status, and date range.
    - Timesheet and effort reports.
    - Milestone slippage report.

Monitoring checklists emphasize dashboards, status reports, and timeline views as key tools in the control phase of a project.[2][4]

4.10 Notifications and alerts

- In-app notifications for:
  - New task assignment.
  - Task status changes.
  - Overdue tasks or milestones.
- Optional email or push notifications (for mobile).
- Configurable notification preferences per user.

Alerts are an important part of proactive monitoring, ensuring deviations are surfaced early for corrective action.[^2]

4.11 Mobile app features (React Native)

- View assigned projects and tasks.
- Submit progress updates (including offline-first option if implemented).
- Log time quickly from mobile.
- Capture and upload photos (e.g., worksite images) as attachments.

Government and field-oriented monitoring systems often rely on mobile apps to capture data from project sites and sync it into central dashboards.[^1]

5. Functional requirements

Functional requirements describe what the system must do.[9][10]

5.1 Authentication and authorization

- **FR-1:** The system shall allow users to authenticate using a unique email/username and password.
- **FR-2:** The system shall enforce role-based access control, restricting operations based on user roles (Admin, ProjectManager, Member, Client).
- **FR-3:** The system shall allow administrators to create, update, deactivate, and assign roles to users.

5.2 Company and project management

- **FR-4:** The system shall allow administrators and project managers to create and maintain company records.
- **FR-5:** The system shall allow project managers to create, update, and close projects, including setting dates, budget, status, and priority.
- **FR-6:** The system shall allow project managers to assign and remove members from projects.

5.3 Milestone and task management

- FR-7: The system shall allow project managers to create milestones with due dates and statuses for each project.
- FR-8: The system shall allow project managers to create tasks with attributes such as dates, status, priority, and estimated hours under a project and optionally under a milestone.
- FR-9: The system shall allow project managers to assign one or more members to a task.
- FR-10: The system shall provide a Kanban board for viewing and updating task statuses.

5.4 Progress updates and tracking

- FR-11: The system shall allow users to submit progress updates at task or project level with percent complete and narrative description.
- FR-12: The system shall automatically calculate project completion percentage based on task or milestone progress.
- FR-13: The system shall maintain a history of progress updates per task and per project.

5.5 Time logging

- FR-14: The system shall allow users to record daily or weekly time entries against tasks.
- FR-15: The system shall provide project managers views and reports of total hours by project, user, and date range.

5.6 Attachments and documents

- FR-16: The system shall allow users with appropriate permissions to upload, view, and download attachments linked to projects and tasks.

5.7 Dashboards and reporting

- FR-17: The system shall display a project dashboard summarizing key indicators, including status, completion percentage, upcoming milestones, and overdue tasks.
- FR-18: The system shall generate standard reports (project status, task summary, timesheet summary) in tabular format and provide export to CSV or PDF when needed.

5.8 Notifications

- FR-19: The system shall generate notifications when a user is assigned a task.
- FR-20: The system shall notify users of overdue tasks or milestones daily until resolved or acknowledged.

5.9 Mobile functionality

- **FR-21:** The mobile app shall allow users to view their assigned tasks and projects.
- **FR-22:** The mobile app shall allow users to submit progress updates and time logs.
- **FR-23:** The mobile app shall allow users to upload photos as attachments to tasks or projects.

6. Non-functional requirements

Non-functional requirements describe how the system must behave with respect to performance, security, usability, and other qualities.[11][12] [^9]

6.1 Performance and scalability

- **NFR-1:** The system shall support at least a defined number of concurrent users (for example 200 active users) with acceptable response times under normal load.
- **NFR-2:** Typical API requests (e.g., fetching project dashboard) should complete within a target response time such as 1–2 seconds under expected load.
- **NFR-3:** The system shall be designed to scale horizontally at the API layer (ASP.NET Core Web API) with stateless services behind a load balancer.

Performance and scalability are key NFRs for project management systems used by large teams or government departments.[^11]

6.2 Availability and reliability

- **NFR-4:** The system should target high availability (for example 99% uptime) during defined business hours.
- **NFR-5:** The system shall tolerate transient failures by using retry mechanisms and graceful error handling at the client level.

Reliability and resilience are typical NFR categories for operational systems.[^11]

6.3 Security

- **NFR-6:** All API communication shall use HTTPS with TLS encryption.
- **NFR-7:** User passwords shall be stored as salted hashes and never in plain text.
- **NFR-8:** The system shall require authentication tokens (such as JWT or cookies with server-side validation) for all protected endpoints.[6][7]
- **NFR-9:** The system shall implement authorization checks at the API layer to prevent privilege escalation.

Security is a critical non-functional requirement and is often categorized separately in NFR guidance.[12][11]

6.4 Usability

- **NFR-10**: The web UI shall be usable on modern desktop and tablet browsers with responsive layouts.
- **NFR-11**: Key workflows (viewing assigned tasks, updating status, logging time) should require a minimal number of clicks/taps.

Usability is an important quality attribute ensuring the system is actually usable by project teams.[^11]

6.5 Maintainability and extensibility

- **NFR-12**: The codebase shall follow layered architecture with separate Domain, Application, Infrastructure, and API projects on the backend, and feature-based structure on the frontend to support modular development and ease of changes.[7][8][^6]
- **NFR-13**: The system shall include automated tests (unit/integration) for critical business logic in the backend.

6.6 Localization and regulatory compliance (optional)

- **NFR-14**: The system should support configuration of date/time formats, number formats, and basic localization for labels if deployed internationally.
- **NFR-15**: The system shall comply with relevant data protection policies and organizational IT security guidelines.

Localization and regulatory compliance are common NFRs in enterprise and government projects.[^11]

7. Database schema

The schema below is relational and can be implemented in SQL Server, PostgreSQL, or another RDBMS. It is aligned with standard ER models for project management systems, extended with progress and time tracking.[13][14]

7.1 Tables and key columns

Table	Key columns (PK, FKs)	Description
Company	CompanyID (PK)	Organization or client owning projects.
User	UserID (PK)	All system users with role and credentials.
Project	ProjectID (PK), CompanyID (FK), ManagerID (FK → User)	Projects under companies.
ProjectMember	ProjectMemberID (PK), ProjectID (FK), UserID (FK)	Mapping between projects and users.
Milestone	MilestoneID (PK), ProjectID (FK)	Milestones belonging to a project.

Table	Key columns (PK, FKs)	Description
Task	TaskID (PK), ProjectID (FK), MilestoneID (FK, nullable)	Tasks under a project and optionally a milestone.
TaskAssignment	TaskAssignmentID (PK), TaskID (FK), UserID (FK)	Mapping between tasks and assigned users.
TimeLog	TimeLogID (PK), TaskID (FK), UserID (FK)	Time entries against tasks.
ProgressUpdate	ProgressUpdateID (PK), ProjectID (FK), TaskID (FK, nullable), UserID (FK)	Structured progress updates for tasks or projects.
Attachment	AttachmentID (PK), ProjectID (FK), TaskID (FK, nullable), UploadedBy (FK → User)	Files linked to projects or tasks.

7.2 Example column definitions (simplified)

Company

- CompanyID (PK, GUID or INT)
- Name (VARCHAR)
- Industry (VARCHAR, nullable)
- Address (VARCHAR, nullable)
- ContactPerson (VARCHAR, nullable)
- ContactEmail (VARCHAR, nullable)

User

- UserID (PK)
- FullName (VARCHAR)
- Email (VARCHAR, unique)
- PasswordHash (VARCHAR)
- Role (ENUM/Admin, ProjectManager, Member, Client)
- Status (ENUM/Active, Inactive)
- CreatedAt, UpdatedAt (TIMESTAMP)

Project

- ProjectID (PK)
- CompanyID (FK → Company.CompanyID)
- ManagerID (FK → User.UserID)
- Name, Description (VARCHAR, TEXT)
- StartDate, EndDate (DATE)
- Budget (DECIMAL, nullable)
- Status (ENUM/Planned, Active, OnHold, Completed, Cancelled)

- Priority (ENUM/High, Medium, Low)
- CreatedAt, UpdatedAt (TIMESTAMP)

ProjectMember

- ProjectMemberID (PK)
- ProjectID (FK → Project.ProjectID)
- UserID (FK → User.UserID)
- MemberRole (VARCHAR)
- AllocationPercent (INT, nullable)

Milestone

- MilestoneID (PK)
- ProjectID (FK)
- Title, Description
- DueDate (DATE)
- Status (ENUM/Pending, InProgress, Completed)

Task

- TaskID (PK)
- ProjectID (FK)
- MilestoneID (FK, nullable)
- Title, Description
- StartDate, EndDate (DATE)
- Status (ENUM/ToDo, InProgress, Blocked, Done)
- Priority (ENUM/Critical, High, Medium, Low)
- EstimatedHours (DECIMAL, nullable)

TaskAssignment

- TaskAssignmentID (PK)
- TaskID (FK)
- UserID (FK)
- AssignedDate (DATE)

TimeLog

- TimeLogID (PK)
- TaskID (FK)
- UserID (FK)
- LogDate (DATE)

- HoursSpent (DECIMAL)
- Description (TEXT, nullable)

ProgressUpdate

- ProgressUpdateID (PK)
- ProjectID (FK)
- TaskID (FK, nullable)
- UserID (FK)
- UpdateDateTime (TIMESTAMP)
- PercentComplete (INT 0–100)
- StatusSummary (TEXT)
- IssuesRisks (TEXT, nullable)

Attachment

- AttachmentID (PK)
- ProjectID (FK)
- TaskID (FK, nullable)
- UploadedBy (FK → User.UserID)
- FileName (VARCHAR)
- FilePath or URL (VARCHAR)
- UploadedAt (TIMESTAMP)

This schema reflects patterns from example ERDs for project management and task management systems.[15][13]

8. ER diagram (textual and Mermaid)

Based on the schema above, the ER diagram includes the following main entities and relationships, similar to typical project/task/employee ER examples.[14][13] [^15]

- Company 1 ——< Project
- User 1 ——< Project (as Manager)
- Project 1 ——< ProjectMember >——1 User
- Project 1 ——< Milestone
- Project 1 ——< Task
- Milestone 1 ——< Task (optional)
- Task 1 ——< TaskAssignment >——1 User
- Task 1 ——< TimeLog, and User 1 ——< TimeLog
- Project 1 ——< ProgressUpdate, Task 1 ——< ProgressUpdate (optional), User 1 ——< ProgressUpdate

- Project 1 ——< Attachment, Task 1 ——< Attachment (optional), User 1 ——< Attachment

A Mermaid ERD representation (for documentation tools that support Mermaid) could be:

```
erDiagram
  COMPANY ||--o{ PROJECT : owns
  USER ||--o{ PROJECT : manages
  PROJECT ||--o{ PROJECT_MEMBER : has
  USER ||--o{ PROJECT_MEMBER : participates

  PROJECT ||--o{ MILESTONE : includes
  PROJECT ||--o{ TASK : contains
  MILESTONE ||--o{ TASK : groups

  TASK ||--o{ TASK_ASSIGNMENT : assigned_to
  USER ||--o{ TASK_ASSIGNMENT : works_on

  TASK ||--o{ TIME_LOG : tracked_by
  USER ||--o{ TIME_LOG : logs

  PROJECT ||--o{ PROGRESS_UPDATE : has
  TASK ||--o{ PROGRESS_UPDATE : details
  USER ||--o{ PROGRESS_UPDATE : submits

  PROJECT ||--o{ ATTACHMENT : stores
  TASK ||--o{ ATTACHMENT : attaches
  USER ||--o{ ATTACHMENT : uploads

  COMPANY {
    int CompanyID PK
    string Name
  }
  USER {
    int UserID PK
    string FullName
    string Email
    string Role
  }
  PROJECT {
    int ProjectID PK
    int CompanyID FK
    int ManagerID FK
    string Name
  }
  PROJECT_MEMBER {
    int ProjectMemberID PK
    int ProjectID FK
    int UserID FK
  }
  MILESTONE {
    int MilestoneID PK
    int ProjectID FK
  }
  TASK {
    int TaskID PK
    int ProjectID FK
```

```
    int MilestoneID FK
  }
  TASK_ASSIGNMENT {
    int TaskAssignmentID PK
    int TaskID FK
    int UserID FK
  }
  TIME_LOG {
    int TimeLogID PK
    int TaskID FK
    int UserID FK
  }
  PROGRESS_UPDATE {
    int ProgressUpdateID PK
    int ProjectID FK
    int TaskID FK
    int UserID FK
  }
  ATTACHMENT {
    int AttachmentID PK
    int ProjectID FK
    int TaskID FK
    int UploadedBy FK
  }
```

9. Module and architecture diagram (logical, React + ASP.NET Core)

A typical layered architecture for a React + React Native + ASP.NET Core system follows a separation of presentation, API, application, domain, and data access, as recommended in ASP.NET Core Clean Architecture examples.[8][6] [^7]

Layers / modules:

- Presentation layer (clients)
  - React web app (manager portal).
  - React Native mobile app (team/field app).
- Web API layer (ASP.NET Core)
  - Controllers, request/response mapping, authentication, and validation.
- Application layer
  - Use-case oriented services and interfaces implementing business logic: create project, assign task, submit progress, etc.
- Domain layer
  - Entities and domain rules (Company, User, Project, Task, etc.).
- Infrastructure layer
  - EF Core DbContext, repository implementations, external services (file storage, email, etc.).
- Database

- Relational DB implementing the schema described earlier.

A simple conceptual module diagram:

```
flowchart LR
  subgraph Clients
    Web[React Web App]
    Mobile[React Native App]
  end

  subgraph API[ASP.NET Core Web API]
    Controllers
    ApplicationServices[Application Services]
    Domain[Domain Entities]
    Infra[Infrastructure Repositories]
  end

  DB[(Relational Database)]

  Web -->|REST/JSON| Controllers
  Mobile -->|REST/JSON| Controllers
  Controllers --> ApplicationServices --> Infra --> DB
  ApplicationServices --> Domain
```

This reflects guidance to keep controllers thin, push business logic into application services, and separate domain models and infrastructure for maintainability and testability.[6][7] [^8]

10. React project structure (web and mobile)

Modern guidance for scalable React applications encourages feature-based or domain-based folder structures where files are grouped by feature instead of file type.[16][17] [18][19]

10.1 React web app structure

A recommended structure for the web client could be:

```
/frontend-web
  package.json
  tsconfig.json
  vite.config.ts or webpack.config.js
  src/
    app/
      App.tsx
      index.tsx
      routes.tsx
    core/
      config/
      constants/
      hooks/
      providers/
      styles/
    features/
      auth/
```

```
        components/
        pages/
        api/
        hooks/
        types.ts
      projects/
        components/
        pages/
        api/
        hooks/
        types.ts
      tasks/
        components/
        pages/
        api/
        hooks/
        types.ts
      reports/
        ...
    components/
      ui/       // shared UI components (Button, Modal, Table, etc.)
      layout/   // Header, Sidebar, Shell
    services/
      httpClient.ts   // axios/fetch wrapper targeting ASP.NET Core Web API
    store/
      index.ts        // Redux/Zustand slices per feature
```

This follows a feature-based architecture where each feature (auth, projects, tasks, reports) is self-contained and uses shared UI and infrastructure modules, as recommended in modern React architecture articles.[17][18] [19][16]

10.2 React Native app structure

The React Native app can mirror the same feature orientation:

```
/mobile
  package.json
  src/
    app/
      App.tsx
      navigation/
        RootNavigator.tsx
    features/
      auth/
      projects/
      tasks/
      timesheets/
    components/
      ui/
      layout/
    services/
      apiClient.ts   // points to ASP.NET Core Web API
```

```
    store/
      ...
```

Using consistent feature names across web and mobile simplifies API integration and shared domain understanding.

11. ASP.NET Core Web API project structure

ASP.NET Core Clean Architecture examples typically recommend separate projects for API, Application, Domain, and Infrastructure to enforce clear dependencies and maintainability.[20][7][8][6]

A practical solution layout:

```
ProjectProgressMonitoring.sln

/backend
  ProjectProgress.Api           // ASP.NET Core Web API (Presentation layer)
  ProjectProgress.Application    // Application/use-case layer
  ProjectProgress.Domain        // Domain entities and logic
  ProjectProgress.Infrastructure // EF Core, repositories, external services

/frontend-web
  ... (React web as above)

/mobile
  ... (React Native as above)
```

11.1 Web API project (`ProjectProgress.Api`)

```
ProjectProgress.Api
  Controllers/
    AuthController.cs
    CompaniesController.cs
    ProjectsController.cs
    TasksController.cs
    ReportsController.cs
  Filters/
  Models/         // Request/response DTOs (if not in Application)
  Program.cs
  appsettings.json
```

- Controllers handle HTTP concerns and delegate to Application services.[20][6]
- DTOs map between HTTP payloads and Application/Domain models.

11.2 Application layer (`ProjectProgress.Application`)

```
ProjectProgress.Application
  Interfaces/
    IAuthService.cs
    IProjectService.cs
    ITaskService.cs
    ITimeLogService.cs
    IProgressService.cs
    ICompanyRepository.cs
    IProjectRepository.cs
    ITaskRepository.cs
    ...
  Services/
    AuthService.cs
    ProjectService.cs
    TaskService.cs
    TimeLogService.cs
    ProgressService.cs
  DTOs/
    ProjectDto.cs
    TaskDto.cs
    ...
  // Optional CQRS pattern
  Features/
    Projects/Commands/
    Projects/Queries/
```

The Application project contains use-case logic and abstractions; it does not depend on EF Core or ASP.NET types, aligning with Clean Architecture principles.[7][8] [^6]

11.3 Domain layer (`ProjectProgress.Domain`)

```
ProjectProgress.Domain
  Entities/
    Company.cs
    User.cs
    Project.cs
    ProjectMember.cs
    Milestone.cs
    Task.cs
    TaskAssignment.cs
    TimeLog.cs
    ProgressUpdate.cs
    Attachment.cs
  Enums/
    ProjectStatus.cs
    TaskStatus.cs
    Priority.cs
  ValueObjects/
  DomainEvents/
```

The Domain project defines core entities and business rules without any infrastructure or framework dependencies.[8][6]

11.4 Infrastructure layer (`ProjectProgress.Infrastructure`)

```
ProjectProgress.Infrastructure
  Persistence/
    ApplicationDbContext.cs
    EntityTypeConfigurations/
      ProjectConfiguration.cs
      TaskConfiguration.cs
      ...
  Repositories/
    CompanyRepository.cs
    ProjectRepository.cs
    TaskRepository.cs
    TimeLogRepository.cs
    ...
  Services/
    EmailService.cs
    FileStorageService.cs
```

- Implements repository interfaces from Application using EF Core.
- Contains other infrastructure services (email, file storage, logging) that the Application layer consumes through abstractions.[6][20] [^7]

The Web API project wires everything together via dependency injection: registering DbContext, repositories, application services, and configuring authentication/authorization.

12. Summary

This specification defines a full set of functional features, non-functional requirements, relational schema, ERD, layered architecture, and React + ASP.NET Core project structures for a Project Progress Monitoring System. The design aligns with widely accepted practices for project management software and modern Clean Architecture in ASP.NET Core, and it can be implemented incrementally per module (auth, projects, tasks, tracking, reporting) for a robust final-year or production-grade application.[13][7] [6][2] [^1]

References

1. Project Monitoring System | Project Tracking & Managing - CSM - Project Monitoring System is an IT solution with the prime objective of facilitating proper monitori...
2. What is project monitoring? Definition, Methods, Process and ... - Project monitoring is defined as the process of tracking, reviewing, and regulating the progress and...
3. What is Monitoring and Control in Project Management? - Track Progress: Monitor the actual implementation of the project along with indicators such as desig...
4. Project monitoring and control: Complete guide (+ checklist) - Monitoring and controlling is the process of regularly observing and tracking the progress of your p...

5. Tutorial: Create an ASP.NET Core app with React in Visual Studio - In this article, you learn how to build an ASP.NET Core project to act as an API backend and a React...
6. Clean Architecture In ASP.NET Core Web API - In this article, you will learn about Clean Architecture in Asp.net Core Web API.
7. Essential Guide to ASP.NET Core Clean Architecture - Implement Clean Architecture in ASP.NET Core for better testability, maintainability, and scalabilit...
8. Clean Architecture in .NET - Code Maze - Clean architecture is an architecture pattern aimed at building applications that we can maintain, s...
9. Non-functional requirement - Wikipedia
10. Nonfunctional Requirements - Who needs function? When managing your next project, make sure the nonfunctional requirements are cl...
11. What are non-functional requirements (NFRs) in project management? - Data shows that inaccurate requirement gathering is one of the key reasons up to 70% of projects fai...
12. Non-Functional Requirements: Tips, Tools, and Examples - Learn what non-functional requirements are, with examples by industry and best practices for buildin...
13. How to Design ER Diagrams for Project Management Software - In this article, we'll explore the process of designing an ER diagram for project management softwar...
14. ER diagram - Project, Task and Employee - Stack Overflow - A project has several tasks. Out of these tasks, an Employee will only be assigned to some of the ta...
15. Task and Project Management System" Entity-Relationship Model ... - Diagram Maker. Make flowchart, software, business diagrams · Chart Maker. Visualize data effortlessl...
16. Scalable React Projects with Feature-Based Architecture - Introduction scaling React apps gets messy quickly as they grow. Flat structures like components/,.....
17. Feature-based React Architecture - Robin Wieruch - How to create a feature-based React architecture that allows large scale applications ...
18. How To Structure React Projects From Beginner To Advanced - In this article I will be covering 3 different folder structures that can be used across various siz...
19. Recommended Folder Structure for React 2025 - DEV Community - Organize your components, hooks, and logic by features (also called domain-based structure). This he...
20. Clean Architecture in ASP.NET Core Web API - Dot Net Tutorials - Learn how Clean Architecture in ASP.NET Core Web API can improve your project's structure and mainta...